

数据源稳定性架构 — 设计文档 v1

0. 现状总览

当前数据依赖图



当前架构问题：

- 1. 每个模块直调外部API — 没有统一的数据获取入口
- 2. 单文件混合多源数据 — sector_daily.json 同时存 THS 和 EM 的数据，指数值不同无法区分
- 3. 无健康监测 — ak(THS)被墙了没人知道，直到用户发现板块数据停在6月1日
- 4. 无自动切换 — 一个源挂了，业务代码返回空数据而不是尝试其他源
- 5. 无告警 — 错误被 except: pass 吞掉，cron日志没人看

外部数据源清单

#	数据类别	数据源	URL/方式	数据内容	稳定性	同类别其他可选源
1	个股K线	mootdx(通达信)	TCP直连218.6.170.47:7709	日K线(复权)	可用	ifzq.gtimg.cn(腾讯K线)
2	个股K线	ifzq.gtimg.cn	web.ifzq.gtimg.cn/appstock/app/fqkline	复权日K线	可用	mootdx
3	板块实时	push2test.eastmoney.com	push2test.eastmoney.com/api/qt/clist/get	板块名称+行情	可用	akshare EM系列
4	板块K线	akshare(THS)	ak.stock_board_industry_index_ths	同花顺板块K线	不可用	push2test(仅今日)
5	板块排行	akshare(THS)	ak.stock_board_industry_summary_ths	板块当日排行	不可用	push2test
6	概念映射	push2test.eastmoney.com	f103接口	个股→概念归属	可用	akshare EM f103

7	实时行情	qt.gtimg.cn(腾讯)	qt.gtimg.cn/q=sh/sz+code	个股/指数实时价	可用	mootdx, sina
8	美股/汇率	hq.sinajs.cn(新浪)	hq.sinajs.cn/list=...	美股+汇率	可用	—
9	热点排行	dq.10jqka.com.cn(同花顺)	dq.10jqka.com.cn/fuyao/hot_list	热门个股Top100	可用	—
10	创新高排名	akshare(THS)	ak.stock_rank_cxg_ths	创新高股票	不可用	无
11	指数K线	akshare(腾讯)	ak.stock_zh_index_daily_tx	指数日K线	可用	web.ifzq.gtimg.cn
12	涨停/情绪	akshare(EM)	ak.stock_zt_pool_em	涨停板池	可用	—
13	财务指标	akshare(EM)	ak.stock_financial_analysis_indicator	ROE/EPS等	可用	—
14	宏观数据	akshare(同花顺)	ak.macro_china_cpi_monthly	CPI/PPI	可用	—
15	行业映射	push2test.eastmoney.com	f100接口	个股→申万行业	可用	akshare(推)
16	AI分析	LLM API	环境变量LLM_API_URL	新闻摘要/标签	条件可用	—

风险矩阵

风险等级	数据类别	影响	当前应对
高	板块排行	每日复盘无排行、主线评分无板块数据	无（卡住）
高	板块K线	概念波动追踪、行业评分无历史数据	无（卡住）
高	创新高排名	趋势候选无新高数据	无
中	个股K线	有备源(mootdx→ifzq.gtimg.cn)	3次重试
中	概念映射	push2test可用，akshare EM f103做备源	有备源
低	实时行情	腾讯 mootdx 新浪三个源可互切	无切源逻辑

业务	数据源	数据源、数据源、数据源、数据源、数据源	数据源、数据源
----	-----	---------------------	---------

1. 背景与问题

现状

3L交易系统依赖于 16 个外部数据源（含内部衍生数据）来驱动全部业务功能。这些数据源来自不同的服务商（通达信、同花顺、东方财富、腾讯、新浪），各有不同的可用性、响应速度和数据格式。

痛点

1. 数据源故障无感知

akshare(同花顺THS) API 在某个时间点被墙，但没有任何告警通知。板块数据停在6月1日，用户直到6月6日手动检查才发现。错误被 `except: pass` 静默吞掉。

2. 单文件混存多源数据导致不可用

`update_sectors()` 把 THS 板块指数和 push2test(东财) 板块指数写入同一个 `sector_daily.json`。两个源的“半导体”指数值不同（THS~5000 vs 东财~2947），混在一起算涨跌幅时得到 +500% / -80% 之类的错误结果。

3. 每类数据只有单一获取路径

业务代码直接调用具体数据源的函数。如果这个源挂了，没有自动降级逻辑。例如 `get_industry_rankings()` 只调 `ak.stock_board_industry_summary_ths()`，THS不可用就返回空。

4. 没有统一的数据健康视图

没有人能回答“当前所有数据源的状态是什么”，没有人知道某个源已经挂了多久。

目标

建立一个自修复、可观测、渐进降级的数据源架构：

- 自修复：源挂了自动切下一顺位，业务不中断
- 可观测：每个源的健康状态实时可见，状态变化自动告警
- 渐进降级：优先用实时数据→降级到缓存数据→降到“数据不可用”提示（不崩页面）

2. 设计思想

2.1 核心理念：数据仓库隔离 + 抽象层统一

为什么必须隔离？

不同源的“半导体”指数值：	
THS(同花顺)：	东财(push2test)：
2026-06-04 close: 5023	2026-06-06 close: 2947
但它们计算的涨跌幅是一致的：	
THS: (5023 - 4988) / 4988 = +0.7%	
东财: (2947 - 2925) / 2925 = +0.75%	

涨跌幅%是跨源共识，绝对值不是。这是整个架构的基石。

所以设计：- 数据仓库层：每个源独立存储，数值不交叉 - 抽象层：只暴露涨跌幅%、排行等跨源通用指标给业务

2.2 方案选择

方案	做法	优点	缺点	结论
A. 多仓库+抽象层	各源独立存储，统一接口切换	隔离干净，切换透明，可观测性强	实现工作量中等	✔ 选这个
B. 单仓库+时间戳	同一文件，加 <code>_source</code> 字			

tag	段区分	改动最少	查询时还得过滤，容易混	×
C. 只用一个源	push2test覆盖全部功能	最简单	push2test没历史K线，且单点故障	×

2.3 设计原则

1. 数据按源隔离 — 不跨源比较绝对值，涨跌幅%是唯一通用指标
2. 调用链有序 — 每类数据有明确优先级链（主源→备源→缓存→降级）
3. 自动切换 — 上层不关心具体源，抽象层负责切
4. 健康监测内建 — 每次调用自动更新源状态，状态变化推送到微信

2.4 屏幕内外

这个架构做什么：- 统一所有外部数据获取的入口和模式 - 每个数据源独立存储到各自文件/空间 - 抽象层提供 `get_xxx()` 接口，内部自动选择可用源 - 逐源健康监测 + 状态变化自动告警

这个架构不做什么：- 不改变现有业务逻辑（`get_sector_rankings()` 返回格式不变） - 不改变 cron 定时任务的结构 - 不改变 frontend API 路由和返回格式 - 不创造新数据（只重新组织数据获取路径）

3. 数据模型

3.1 文件结构

```

data/
├── sources/                                # ← 新增：每个数据源独立目录
│   ├── em/                                # 东财仓（push2test）
│   │   ├── sector_daily.json              # 行业/概念每日数据（change_pct）
│   │   └── concept_map.json               # 概念映射
│   ├── ths/                                # 同花顺仓（akshare）
│   │   ├── sector_daily.json              # 行业/概念日K线（历史）
│   │   └── stock_cxg.json                 # 创新高列表
│   ├── tencent/                           # 腾讯仓
│   │   └── realtime_cache.json            # 实时行情缓存
│   └── mootdx/                             # 通达信仓
│       ├── klines/                        # 个股K线（已有）
│       └── source_health.json              # ← 新增：数据源健康状态

```

3.2 各数据源仓库格式

EM仓 (push2test) — 板块每日数据：

```

{
  "last_updated": "20260606",
  "industries": {
    "半导体": {
      "date": "20260606",
      "change_pct": 2.45,
      "close": 2947.18,
      "prev_close": 2876.72,
      "open": 2900.46,
      "high": 2949.29,
      "low": 2896.11,
      "volume": 128833
    }
  },
  "concepts": {}
}

```

关键： `change_pct` 来自于东财自己的涨跌幅计算（f3），不依赖历史数据，直接可用。

THS仓 (akshare) — 板块历史K线：

```
{
  "last_updated": "20260604",
  "version": "ths-board-kline-v1",
  "industries": {
    "半导体": [
      {"date": "20260401", "open": 4800, "close": 4850, ...},
      {"date": "20260604", "open": 5010, "close": 5023, ...}
    ]
  }
}
```

关键：THS 仓库只保留 THS 源自己的数据，不混入东财数据。 `change_pct` 由抽象层从相邻K线计算。

3.3 健康监测数据模型

```
{
  "sources": {
    "mootdx": {
      "status": "UP",
      "last_ok": "20260606 17:00:23",
      "last_fail": "",
      "fail_count": 0,
      "total_calls": 89234,
      "success_rate_pct": 99.8
    },
    "push2test_em_sector": {
      "status": "UP",
      "last_ok": "20260606 17:00:21",
      "last_fail": "",
      "fail_count": 0,
      "total_calls": 325,
      "success_rate_pct": 100.0
    },
    "akshare_ths": {
      "status": "DOWN",
      "last_ok": "20260601 17:00:15",
      "last_fail": "20260606 18:00:00",
      "fail_count": 48,
      "total_calls": 52,
      "success_rate_pct": 7.7,
      "last_error": "ConnectionError: Remote end closed connection"
    },
    "akshare_em_zt_pool": {
      "status": "UP",
      "last_ok": "20260606 17:00:25",
      "last_fail": "",
      "fail_count": 0,
      "total_calls": 128,
      "success_rate_pct": 100.0
    }
  },
  "transitions": [
    {"time": "20260606 18:00:00", "source": "akshare_ths", "from": "UP", "to": "DOWN", "reason": "连续3次连接失败"},
    {"time": "20260606 18:00:30", "alert_sent": true}
  ]
}
```

3.4 数据流



4. 系统设计

4.1 架构总览



4.2 核心函数：_call_with_failover

这是抽象层的核心引擎：

```
def _call_with_failover(data_type, args, chain, fallback=None):  
    """统一的故障切换调用模式
```

```

Args:
    data_type: 数据类别标识（用于告警和健康监测）
    chain: [(源名, 库路径, 获取函数), ...]
    fallback: 可选兜底值

Returns:
    获取到的数据

Raises:
    DataUnavailableError: 所有源均不可用时
"""
for source_name, repo_path, fetch_fn in chain:
    health = get_source_health(source_name)
    if health['status'] == 'DOWN' and health['fail_count'] >= 3:
        # 已标记DOWN的源跳过（除非连续成功，自动恢复）
        continue

    try:
        data = fetch_fn(repo_path, *args)
        report_success(source_name)
        return data
    except Exception as e:
        report_failure(source_name, e)
        log.error(f'[数据源] {data_type} → {source_name} 失败: {e}')
        continue

# 所有源失败 → 尝试fallback
if fallback:
    return fallback

# 无fallback → 告警 + 抛异常
alert(f'⚠ 数据源全部不可用: {data_type}')
raise DataUnavailableError(data_type)

```

4.3 各数据类目的调用链定义

```

DATA_SOURCE_CHAINS = {
    'sector_ranking_industry': [
        ('em_sector', 'sources/em/sector_daily.json',
         lambda path, date: ...读取industries, 取change_pct),
        ('ths_sector', 'sources/ths/sector_daily.json',
         lambda path, date: ...从klines计算change_pct),
    ],
    'sector_ranking_concept': [
        ('em_sector', 'sources/em/sector_daily.json', ...),
        ('ths_sector', 'sources/ths/sector_daily.json', ...),
    ],
    'sector_klines': [
        ('ths_sector', 'sources/ths/sector_daily.json',
         lambda path, name: ...返回历史K线),
        ('em_sector', 'sources/em/sector_daily.json',
         lambda path, name: ...返回单日K线(只有今日)),
    ],
    'stock_realtime': [
        ('tencent', 'qt.gtimg.cn', lambda code: ...),
        ('mootdx', 'quotes.factory', lambda code: ...),
        ('sina', 'hq.sinajs.cn', lambda code: ...),
    ],
    'concept_map': [
        ('em_sector', 'sources/em/concept_map.json', ...),
        ('cache', 'data/concept_map.json', ...),
    ],
}

```

```
'stock_new_high': [
    ('ths_cxg', 'sources/ths/stock_cxg.json', ...), # 待恢复
    ('cache', 'data/latest_scan_result.json', ...), # 用缓存
],
}
```

4.4 健康监测 + 告警

每次 `_call_with_failover` 的执行都会自动更新健康表：

事件	动作
调用成功	<code>fail_count</code> 置 0；如果之前是 DOWN 且现在连续成功3次 → 自动恢复为 UP
调用失败	<code>fail_count</code> +1；连续3次 → 标记 DOWN，记录时间 + 错误信息
DOWN 标记	生成告警消息，推送微信
恢复 UP	生成恢复消息，推送微信
持续 DOWN	不重复发（同源同状态只发一次）

4.5 迁移策略

阶段一：建基础设施（当前） - 创建 `sources/` 目录结构 - 实现 `data_source.py`（抽象层核心） - 实现 `source_health.py`（健康监测） - 将 `push2test` 数据写入 EM 仓（独立文件，不再混入 `sector_daily.json`）

阶段二：逐个模块切换 - 板块排行 → 切到 `data_source.get_sector_rankings()` - 板块K线 → 切到 `data_source.get_sector_klines()` - 实时行情 → 切到 `data_source.get_stock_realtime()` - 概念映射 → 切到 `data_source.get_concept_map()` - 创新高 → 切到 `data_source.get_new_high_list()`

阶段三：清理旧代码 - 移除 `update_sectors()` 中的直接 API 调用 - 移除旧的直调模式 - 删除空转的 `akshare(THS)` 备源代码

5. 执行计划

详见：[数据源稳定性架构 — 执行计划](#)

6. 附录

6.1 替代方案

方案A：单仓库+源标记 在现有 `sector_daily.json` 里加 `_source` 字段标记数据来源。查询时按源过滤。 - 优点：改动最少 - 缺点：文件越来越大，查询逻辑复杂，历史数据和实时数据混在一起，容易误用 - 结论：✖ 否决

方案B：全量用 `push2test` 放弃 `akshare(THS)`，只用 `push2test` 作为板块唯一源。 - 优点：最简单 - 缺点：`push2test` 没有历史K线，概念板块名匹配率99%（不是100%），单点故障 - 结论：✖ 否决（但可作为第一优先源）

6.2 开放问题

- 1. **THS 旧缓存数据处理**：当前 `sector_daily.json` 中的 THS 旧缓存（约37个有历史数据的板块）如何迁移到 THS 仓库？数据格式是否兼容？
- 2. **创新高数据源**：`akshare stock_rank_cxg_ths` 被墙后，创新高排名数据没有替代源。是否可以 从个股K线计算新高？
- 3. **概念板块名称不匹配**：`push2test` 和 THS 的概念板块名字不同（如“芯片概念” vs “芯片”），匹配率99%。剩下的1%如何处理？
- 4. **历史K线积累**：EM 仓只有今日数据，没有历史K线。新板块需要时间积累历史。期间区块历史趋势显示靠谁？

6.3 文件清单

```
新增：
server/backend/services/data_source.py      # 抽象数据层
server/backend/services/source_health.py     # 健康监测
data/sources/em/sector_daily.json           # EM仓(板块每日)
data/sources/em/concept_map.json            # EM仓(概念映射)
```



```
data/sources/thz/sector_daily.json      # THS仓(板块K线)
data/source_health.json                 # 健康状态

修改：
server/backend/core/update_stock_data.py  # 数据写入改为独立仓库
server/backend/core/data_layer.py        # 新增抽象层入口
server/backend/services/market_service.py # 改用 data_source 接口
server/backend/services/review_service.py # 改用 data_source 接口
server/backend/services/monitor_service.py # 改用 data_source 接口
```

6.4 变更日志

版本	日期	变更内容
v1	2026-06-06	初稿